- **Duplicates and repeated records. (Yes/No, how you found them - visual check, ID duplication check?) - YES**
  How we found them: an ID duplication check, then (after dropping ID) a full-row duplicate check.
  df["ID"].nunique()
  df.duplicated().sum()

  Result: all 30,000 IDs are unique, so no customer is repeated by ID. With ID removed, 35 rows (0.12%) are exact copies of another row. Most of them are inactive accounts where every bill and payment is 0.

  Action: removed with the code below, leaving 29,965 rows. The index was reset so the rows are numbered without gaps.
  df.drop_duplicates(inplace=True)
  df.reset_index(drop=True, inplace=True)

  ● Incorrect data types. (eg., a numeric variable such as age having a categorical type) - Partly
  How we found them: checked the stored type of every column.
  df.dtypes

  Result: all 25 columns are stored as int64. The numeric variables (LIMIT_BAL, AGE, BILL_AMT1–6, PAY_AMT1–6) have the correct type. SEX, EDUCATION, and MARRIAGE are categorical variables stored as numeric codes, and PAY_1–PAY_6 are ordinal repayment-status codes.

  Action: SEX, EDUCATION, and MARRIAGE will be treated as categories and encoded. They will not be used as numbers.

- **Invalid, impossible or inconsistent values. - YES (undocumented codes only)**
  How we found them: compared category frequencies against the UCI codebook, and ran range checks on the numeric columns.

  df["EDUCATION"].value_counts().sort_index()
  df["MARRIAGE"].value_counts().sort_index()
  df["AGE"].min(), df["AGE"].max(), df["LIMIT_BAL"].min()
  (df[column] < 0).sum() # for each BILL_AMT and PAY_AMT column

  Result: EDUCATION has codes 0 (14 rows), 5 (280) and 6 (51), and MARRIAGE has code 0 (54 rows). None of these are defined in the documentation. The PAY_ codes -2 and 0 are also undocumented, but they are too common to be errors. Of the 2,750 clients with PAY_1 = -2, 1,856 still have a bill, so we did not reinterpret the code. There are no impossible values: AGE is between 21 and 79, LIMIT_BAL is at least 10,000, and no payment is negative. BILL_AMT has 590–688 negative values per month. These are credit balances (overpayments), not errors.

Action: merged EDUCATION 0, 5, 6 into 4 (Others) and MARRIAGE 0 into 3 (Others). Kept the PAY_ codes and the negative bills as they are.

● Irrelevant columns. (e.g., IDs) - YES
How we found them: the ID check above. ID has one unique value per row (30,000).

Result: ID is only a row number and carries no information about the customer.

Action: dropped with df.drop(columns=["ID"]).

● Constant or near-constant variables. - NO
How we found them: calculated the share of the most common value in every column. df[column].value_counts(normalize=True).max() * 100

Result: the most common value never covers more than 60.4% of a feature (SEX). Most columns are well below that. No column is close to a near-constant level such as 95%. The target is 77.9% non-default, which is class imbalance rather than a constant feature.

Action: nothing removed.

● High-cardinality categories. (e.g., a feature called country having 200 different countries) - NO
How we found them: counted the distinct values in each categorical column.

df[["SEX", "EDUCATION", "MARRIAGE", "PAY_1", ..., "PAY_6"]].nunique()

Result: SEX has 2 values, EDUCATION 7 (4 after merging), MARRIAGE 4 (3 after merging) and PAY_1–PAY_6 10–11 each.

Action: no grouping needed.